Studio Nx

JS 06 — isLoggedIn()

2026-10-12

JS 06 — `isLoggedIn()`

항목	내용
파일	`studionx/frontend/static/js/core/auth.js`
작업	P1-02 프론트 공통 JS (PRD 01 FR-105)
작성	Claude (AI), 2026-10-06, 사용자 검토
이전 · 다음	05 `parseLoginHash()` · 07 `requireLogin()`

1. 하는 일

지금 로그인 상태인지를 `true` 또는 `false`로 알려 줍니다. 토큰이 없으면 리프레시 쿠키로 다시 받아 본 뒤 판단합니다.

2. 언제 쓰나

상황	부르는 곳
로그인이 필요한 페이지(마이페이지, 내 프로젝트, 회원 정보, 시퀀서)에 들어올 때	07 `requireLogin()`이 부르고, `false`이면 로그인 화면으로 보냅니다
페이지를 열 때 맨 위 메뉴 줄(헤더) 오른쪽에 어떤 버튼을 보여 줄지 정할 때	`auth-ui.js`의 `initAuthState()`(P1-07)가 부릅니다. 로그인 상태면 **My Page, My Projects** 버튼을, 아니면 **Sign In, Join Us** 버튼을 보여 줍니다
`home.html`을 열 때	이미 로그인 상태라면 `#login` 모달을 열지 않습니다 (P1-07)

헤더 버튼이 바뀌는 예

헤더는 홈, 가입, 마이페이지 등 모든 페이지 맨 위에 똑같이 들어가는 메뉴 줄입니다 (`static/partials/header.html`). 오른쪽 버튼 네 개가 모두 HTML에 들어 있고, 이 함수의 결과에 따라 두 개만 보이게 합니다.

로그인 상태	보이는 버튼	숨기는 버튼
로그인하지 않음 (`false`)	Sign In, Join Us	My Page, My Projects
로그인함 (`true`)	My Page, My Projects	Sign In, Join Us

버튼에는 `data-auth-show="out"`(로그인하지 않았을 때 보임) 또는 `data-auth-show="in"`(로그인했을 때 보임) 표시가 붙어 있습니다.

로그인 상태를 판단하는 기준

서버에는 "로그인 중"이라는 기록이 없습니다. 브라우저가 가진 토큰으로 판단합니다.

브라우저 상태	판단
메모리에 Access Token이 있습니다	로그인 상태입니다
Access Token은 없지만 리프레시 쿠키가 유효합니다 (페이지를 옮긴 직후, 자동 로그인)	재발급에 성공하면 로그인 상태입니다
둘 다 없거나 리프레시 쿠키가 만료되었습니다	로그인하지 않은 상태입니다

3. 코드

import (파일 맨 위)

```
import { api, clearAccessToken, getAccessToken, refreshAccessToken } from './api.js';
```

`auth.js` 맨 위의 import 줄입니다. 이 함수는 이 중에서 **`getAccessToken`과 `refreshAccessToken`**을 씁니다 (01).

함수

```
export async function isLoggedIn() {
  if (getAccessToken()) return true;
  return refreshAccessToken();
}
```

문법 풀이

import 문법	뜻
`import { a, b } from './api.js'`	`api.js`가 `export`한 것 중 `a`와 `b`를 이름 그대로 가져옵니다. `'./'`는 같은 폴더라는 뜻입니다. 브라우저는 `<script type="module">`일 때만 `import`를 읽습니다

식	뜻
`async function`	결과를 Promise로 돌려주는 함수입니다. 부르는 쪽은 `await isLoggedIn()`으로 `true` 또는 `false`를 받습니다
`if (getAccessToken())`	토큰 문자열이 있으면 참이고, `null`이면 거짓입니다
`return refreshAccessToken();`	재발급 결과(`true` 또는 `false`)를 그대로 돌려줍니다. `async` 함수 안에서 Promise를 돌려주면 그 결과가 이 함수의 결과가 됩니다

4. 흐름

1. 메모리에 Access Token이 있으면 서버에 묻지 않고 `true`를 돌려줍니다.
2. 없으면 `refreshAccessToken()`을 부릅니다 (01).
3. 재발급에 성공하면 `true`, 실패하면 `false`를 돌려줍니다.

5. 왜 이렇게

선택	이유
토큰이 있으면 바로 `true`	같은 페이지 안에서 여러 번 불려도 서버 요청이 생기지 않습니다
토큰의 만료 시간은 확인하지 않음	만료된 토큰으로 API를 부르면 `apiFetch()`가 401을 받고 재발급한 뒤 다시 보냅니다. 같은 처리를 두 곳에 두지 않기 위해서입니다
재발급은 `refreshAccessToken()`에 맡김	헤더 버튼을 정하는 코드와 `requireLogin()`이 페이지를 열 때 동시에 불러도 재발급 요청은 한 번만 나갑니다 (01의 진행 중인 Promise 공유)
실패해도 예외를 던지지 않음	네트워크 오류도 "로그인하지 않음"으로 봅니다. 부르는 쪽은 `true`와 `false`만 확인하면 됩니다

6. 시험 (fetch를 가짜로 바꿔서 시험)

경우	결과
메모리에 토큰이 있음	`true`이고, 서버 요청은 0번입니다
토큰은 없고 리프레시 쿠키가 유효함	`true`이고, 재발급을 1번 하며, 새 토큰을 저장합니다
토큰도 리프레시 쿠키도 없음 (401)	`false`이고, 토큰이 없습니다
네트워크 오류	`false`이고, 예외는 나지 않습니다
동시에 두 번 호출 (헤더 버튼 정하기 + `requireLogin`)	둘 다 `true`이고, 재발급은 1번입니다